Experiment-3

What is System Call?

A **system call** is the programmatic way a computer program requests a service from the operating system's kernel. It acts as the secure bridge between standard software applications and the physical hardware of the machine.

Program vs process

Program (Passive): A program is just a static file containing a set of instructions and data sitting on your hard drive (like a .exe, a compiled C++ binary, or an index.js script). It does absolutely nothing until you launch it.

Process (Active): A process is an executing instance of a program. When you double-click a program or run it from the terminal, the Operating System loads those instructions into RAM, assigns it a Process ID (PID), and the CPU starts executing it.

Listing of all Process System calls:

fork()	exec()	exit()
kill()	wait()	getpid()
getppid()	getuid()	nice()
open()	read()	write()

Aim: Write a program to demonstrate process system calls pid(), ppid(), wait(), sleep(), exit().

Description: This program uses the fork() system call for process creation and if-else conditional statements to determine the execution flow for the parent and child processes. Synchronization is achieved using the wait() and sleep() functions to pause execution, while printf is used alongside getpid() and getppid() to display the process IDs and termination status.

C Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }
    else if(pid == 0) {
        printf("Child PID is: %d\n", getpid());
        printf("Parent's PID (PPID) is: %d\n", getppid());
        sleep(3);
        printf("Exiting now with status 1.\n");
        exit(1);
    }
   else {
        printf("parent process.\n");
        printf("Parent PID is: %d\n", getpid());
        printf("My Parent's PID (PPID) is: %d\n", getppid());
        printf("I created a child process with PID: %d\n", pid);
        printf("Waiting for the child process to terminate...\n");
        wait(NULL);
        printf("Parent process exiting now.\n");
        exit(0);
    }
    return 0;
}
```

Output:



Aim: Write a program to demonstrate Orphan Process.

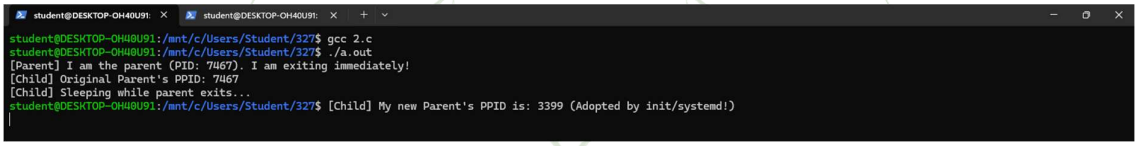
Description: This program demonstrates the creation of an **orphan process** using the fork() system call. The parent process intentionally terminates immediately using exit(0), leaving the child process running in the background while suspended by sleep(3). The child uses getppid() before and after the pause to illustrate that once its original parent dies, it is adopted by the system's init or systemd process, which assigns it a new parent process ID.

C Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(){
        pid_t pid;
        pid = fork();
        if(pid < 0){
                perror("fork Failed");
                exit(1);
        }
        else if (pid == 0) {
        printf("[Child] Original Parent's PPID: %d\n", getppid());
        printf("[Child] Sleeping while parent exits...\n");
        sleep(3);

        printf("[Child] My new Parent's PPID is: %d (Adopted by init/systemd!)\n",
getppid());
        exit(1);
    }
    else {
        printf("[Parent] I am the parent (PID: %d). I am exiting immediately!\n",
getpid());
        exit(0);
    }
    return 0;
}
```

Output:



Aim: Write a program to demonstrate Zombie Process.

Description: This program demonstrates the creation of a **zombie process** using the fork() system call. The child process terminates immediately using exit(0), while the parent process pauses execution using sleep(20) without ever calling wait(). Because the parent fails to read the child's exit status, the terminated child remains stuck in the system's process table as a defunct "zombie" until the parent finally wakes up and exits.

C Program:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t child_pid = fork();

    if (child_pid > 0) {
        printf("Parent process (PID: %d) is running.\n", getpid());
        printf("Child process (PID: %d) was created.\n", child_pid);

        // The parent sleeps for 20 seconds WITHOUT calling wait().
        // During this time, the terminated child becomes a zombie.
        printf("Parent is sleeping for 20 seconds...\n");
        printf("--> Open another terminal and run: ps aux | grep 'Z'\n");

        sleep(20);

        printf("Parent woke up and is exiting.\n");
    }
    else if (child_pid == 0) {
        printf("Child process (PID: %d) is terminating immediately.\n", getpid());
        exit(0);
    }
    else {
        perror("Fork failed");
        return 1;
    }

    return 0;
}
```

Output:

```
student@DESKTOP-OH40U91:/mnt/c/Users/Student/327$ gcc 3.c
student@DESKTOP-OH40U91:/mnt/c/Users/Student/327$ ./a.out
Parent process (PID: 7725) is running.
Child process (PID: 7726) was created.
Parent is sleeping for 20 seconds...
--> Open another terminal and run: ps aux | grep 'Z'
Child process (PID: 7726) is terminating immediately.
Parent woke up and is exiting.
student@DESKTOP-OH40U91:/mnt/c/Users/Student/327$
```

Aim: Write a program to demonstrate exec().

Description: This program demonstrates process creation and program execution using the fork() and exec() family of system calls. The parent process creates two child processes

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid1, pid2;
    pid1 = fork();
    if (pid1 == 0) {
    printf("%d", pid1);
        printf("=== Child 1: Running execv ===\n");
        char *args[] = {"/bin/ls", "-a", NULL};
        execv("/bin/ls", args);
        perror("execv failed");
        exit(1);
    }
    waitpid(pid1, NULL, 0);
    printf("\n");
    pid2 = fork();
    if (pid2 == 0) {
        printf("%d", pid2);
        printf("=== Child 2: Running execl ===\n");
        execl("/bin/ls", "ls", "-l", NULL);
        perror("execl failed");
        exit(1);
    }
    waitpid(pid2, NULL, 0);
    printf("\nParent process finished successfully.\n");
    return 0;
}
```

sequentially, synchronizing their execution by waiting for each to finish using waitpid(). The first child replaces its process image using execv() to run the ls -a command, while the second child uses execl() to execute the ls -l command.

Output:



Aim: Write a program to demonstrate setuid(), getuid().

Description: This program utilizes the getuid() system call to retrieve and display the real user ID of the current process. It then attempts to change the user's privileges by setting a new user ID using the setuid() function. An if-else conditional statement is used alongside perror() to determine and output whether the privilege modification was successful or if it failed.

C Program:



Output:

